

Learning objectives

In this lesson you will learn to:

- Define what is meant by the terms 'low-level language' and 'high-level language'
- Explain why each processor has its own unique instruction set
- Describe how writing a program in a low-level language differs from writing one in a high-level language
- Compare features of low-level and high-level languages and identify tasks for which each is best suited.

For more information and additional student activities see Topic 3.3 of the student book.

Instructions

- An **instruction set** is all the different instructions a microprocessor (CPU) can execute.
- The instructions are stored in memory as binary patterns.
- The instructions are fetched, decoded and executed by the CPU.

Machine code

- The binary patterns, representing a program, are called **machine code**.
- Machine code sits as close to the hardware as possible.
- What does this program, written in machine code, do?

0110100100100101010101011111010101100101011010110101010

- While it is quite simple for the CPU to execute, it's quite a challenge for humans to understand.

Assembly code

- As with machine code, **assembly code** sits very close to the hardware.
- One line of assembly code generates one instruction in machine code.
- What does this program, written in assembly code, do?

```
MOV  x0,  #12
MOV  x1,  #8
ADD  x2,  x0,  x1
```

- This is easier to understand than binary, as the **mnemonics** (short abbreviations) are similar to English.

Define 'low-level' programming language

- Sits very close to the computer's hardware
- Either machine code or assembly language
- Consists of binary codes or simple mnemonics that represent the binary codes.

Define 'high-level' programming language

- Abstracted away from the hardware
- The programmer does not need to understand 'how' the computer works
- Uses English-like statements
- Requires a translator to convert the **source code** to machine code
- Some examples are Python, Java, JavaScript, C++ and C.

What does this Java program output?

```
1 // Java Program
2 int time = 22;
3 if (time < 10) {
4     System.out.println("Good morning.");
5 } else if (time < 20) {
6     System.out.println("Good day.");
7 } else {
8     System.out.println("Good evening.");
9 }
```

Each microprocessor has its own unique instruction set

- Microprocessor architecture is designed with a limited set of electronic components.
- An instruction set is an abstraction that defines the behaviours of these specific electronic components.
- For example, `MOV x0, #12`, is the abstract instruction that causes the value of 12 to be moved into register number 0.
- For each allowable and defined behaviour, there is an abstract instruction.

Each microprocessor with a unique set of components requires a different unique set of instructions.

Compare features

Low-level

- Mnemonics
- One line generates one instruction in machine code
- Microprocessor specific
- A very limited range of instructions is available – every task, even the simplest, has to be built up from the smallest steps
- There are no strings, integers or real numbers, just binary, so you have to decide how to represent your data; you also have to decide and manage where it is stored in memory.

High-level

- English-like keywords
- One line generates multiple instructions in machine code
- Very abstracted instructions that are portable across different architectures, wherever there is a translator
- Complex programming constructs (if, for) available for working at a more abstracted level
- Enforced data types help manage storage.

Writing programs differs

Low-level

- You have to learn the mnemonics
- Mnemonics are more difficult to read and understand without lots of practice
- You have to construct complex actions (if, for) from smaller building blocks
- You have to know about the architecture of the microprocessor, e.g. registers
- Debugging can be more difficult because there are more small instructions to trace
- Allows the programmer to highly tune code for speed of execution.

High-level

- English-like keywords make it easier to read and understand source code
- Very abstracted instructions (if, for) mean the programmer doesn't need to know how the computer works
- Usually quicker to write functional programs, especially if they are large and complex
- Usually will have more choice of development tools, such as IDEs.

Tasks for which each is best suited

Low-level

- Wherever code must be quick and efficient
- Wherever no high-level translator exists for a microprocessor
- Embedded systems
- Device drivers
- Real-time systems
- Parts of operating systems.

High-level

- Where source code must be portable across microprocessor architectures
- Games
- Office productivity software
- Operating systems
- Phone apps.

Wrap up: you have learned how to...

- ✓ Define what is meant by the terms 'low-level language' and 'high-level language'
 - Low-level sits close to the hardware and is either binary machine code or assembly code
 - High-level is more abstracted as it hides the details of the microprocessor
- ✓ Explain why each processor has its own unique instruction set
 - Each microprocessor must have its behaviours defined in terms of the electronics inside of it.

and you have learned how to...

- ✓ Describe how writing a program in a low-level language differs from writing one in a high-level language
 - Low-level uses mnemonics for each small step of a task; high-level uses English-based keywords that indicate multiple steps
- ✓ Compare features of low-level and high-level languages and identify tasks for which each is best suited
 - Tasks that require addressing hardware are suitable for low-level languages. All other tasks can usually be done in a high-level language.